Chapter 6.
I/O Multiplexing: The select and poll Functions

When the TCP client is handling two inputs at the same time:

- standard input
- TCP socket

- problem is encountered

  when the client was blocked in a call to **fgets** (on standard input) and the server process was killed.

  The server TCP correctly sent a FIN to the client TCP, but since the client process was blocked reading from standard input, it never saw the EOF until it read from the socket (possibly much later).

We want to be notified if one or more I/O conditions are ready (i.e., input is ready to be read, or the descriptor is capable of taking more output).

This capability is called **I/O multiplexing** and is provided by the **select** and **poll** functions

I/O multiplexing is typically used in networking applications in the following scenarios:

- When a client is handling multiple descriptors (normally interactive input and a network socket)
- When a client to handle multiple sockets at the same time (this is possible, but rare)
- If a TCP server handles both a listening socket and its connected sockets
- If a server handles both TCP and UDP
- If a server handles multiple services and perhaps multiple protocols

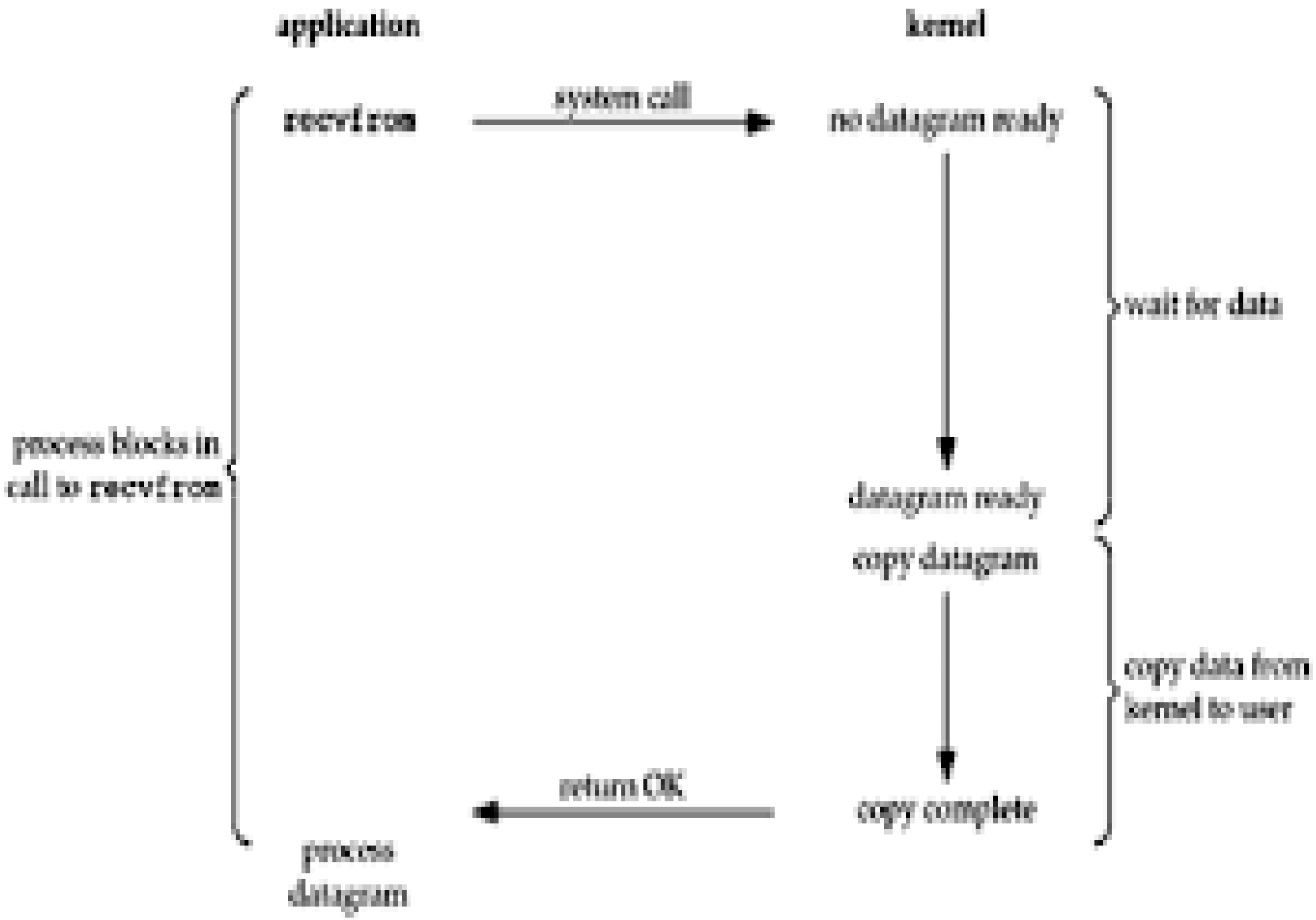
I/O Models

- blocking I/O
- nonblocking I/O
- I/O multiplexing (select and poll)
- signal driven I/O (SIGIO)
- asynchronous I/O (the POSIX aio_ functions)

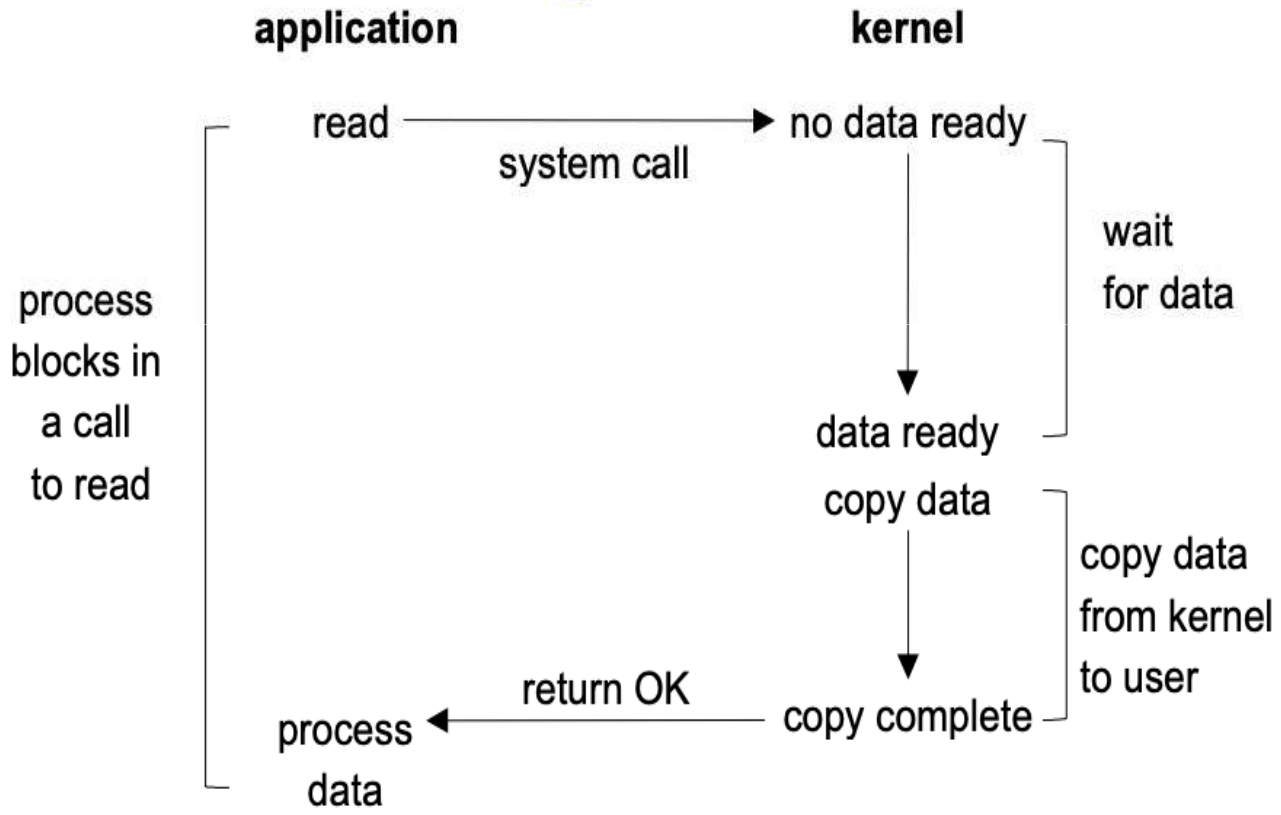
There are normally two distinct phases for an input operation:

- Waiting for the data to be ready. This involves waiting for data to arrive on the network. When the packet arrives, it is copied into a buffer within the kernel.
- Copying the data from the kernel to the process. This means copying the (ready) data from the kernel's buffer into our application buffer

Blocking I/O Model


application
kernel
recvfrom
system call
no datagram ready
wait for data
process blocks in call to recvfrom
datagram ready
copy datagram
copy data from kernel to user
return OK
copy complete
process datagram

Blocking I/O Model

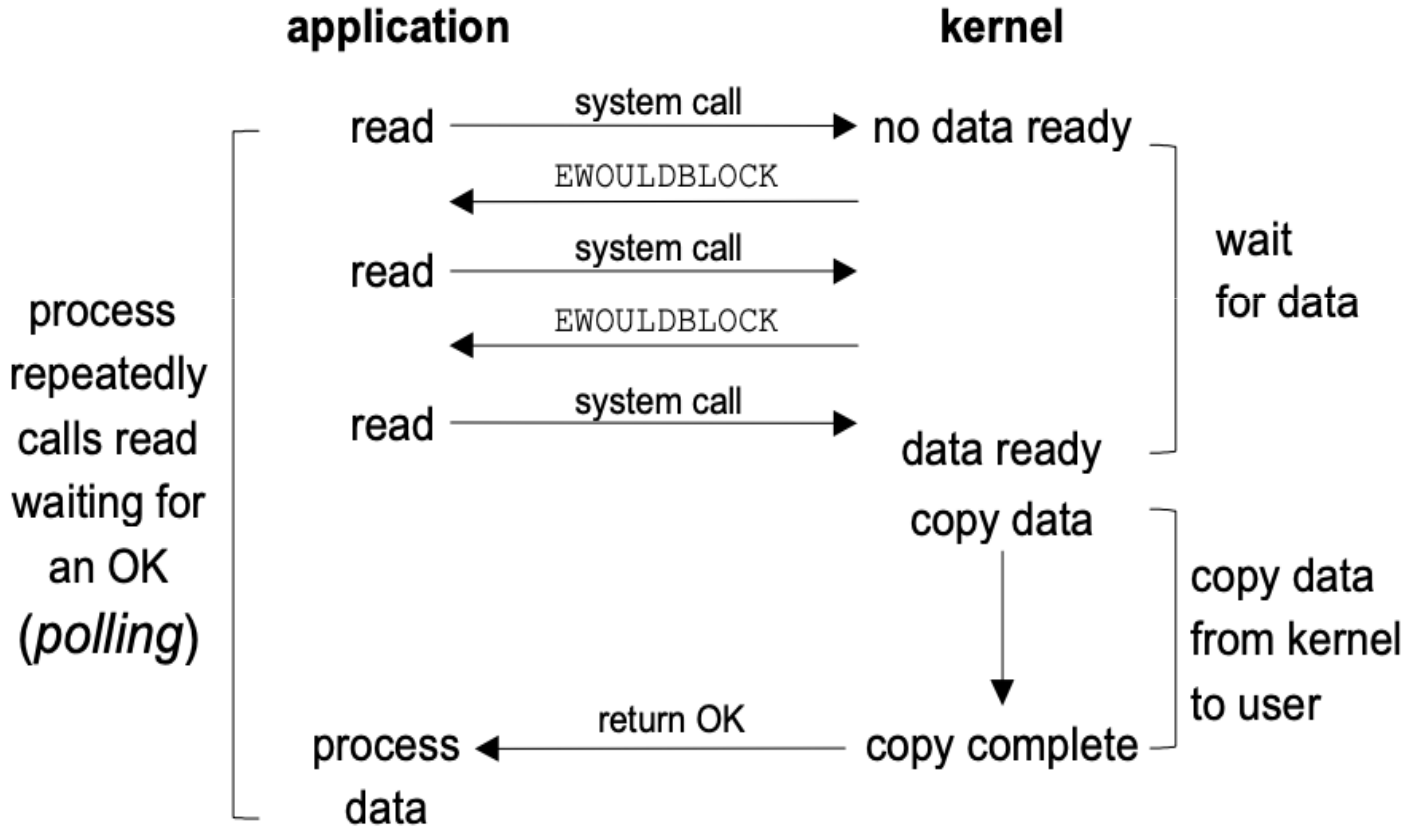


- UDP is used for this example.

  The concept of data being "ready" to read is simple: either an entire datagram has been received or it has not.

- recvfrom () system call is used to differentiate between our application and the kernel
- The process calls recvfrom () and the system call does not return until the datagram arrives and is copied into our application buffer, or an error occurs.
- The most common error is the system call being interrupted by a signal.
- We say that the process is blocked the entire time from when it calls recvfrom () until it returns.
- When recvfrom () returns successfully, our application processes the datagram.

Nonblocking I/O Model

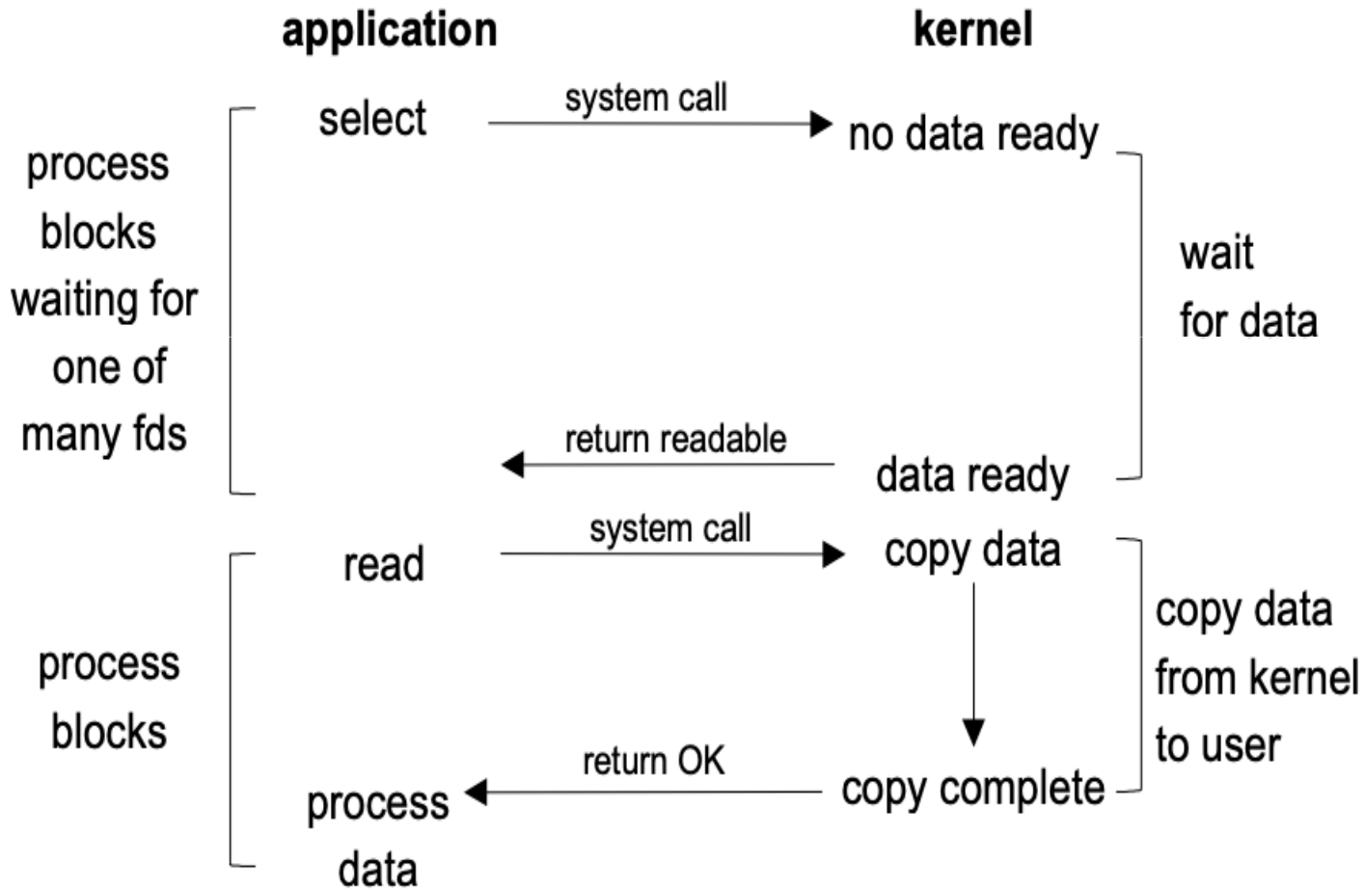


- When a socket is set to be nonblocking, we are telling the kernel

"when an I/O operation that I request cannot be completed without putting the process to sleep, do not put the process to sleep, but return an error instead".

- For the first three recvfrom, there is no data to return and the kernel immediately returns an error of EWOULDBLOCK.
- For the fourth time we call recvfrom, a datagram is ready, it is copied into our application buffer, and recvfrom returns successfully. We then process the data.
- When an application sits in a loop calling recvfrom on a nonblocking descriptor, it is called **polling**.
- The application is continually polling the kernel to see if some operation is ready.

I/O Multiplexing Model

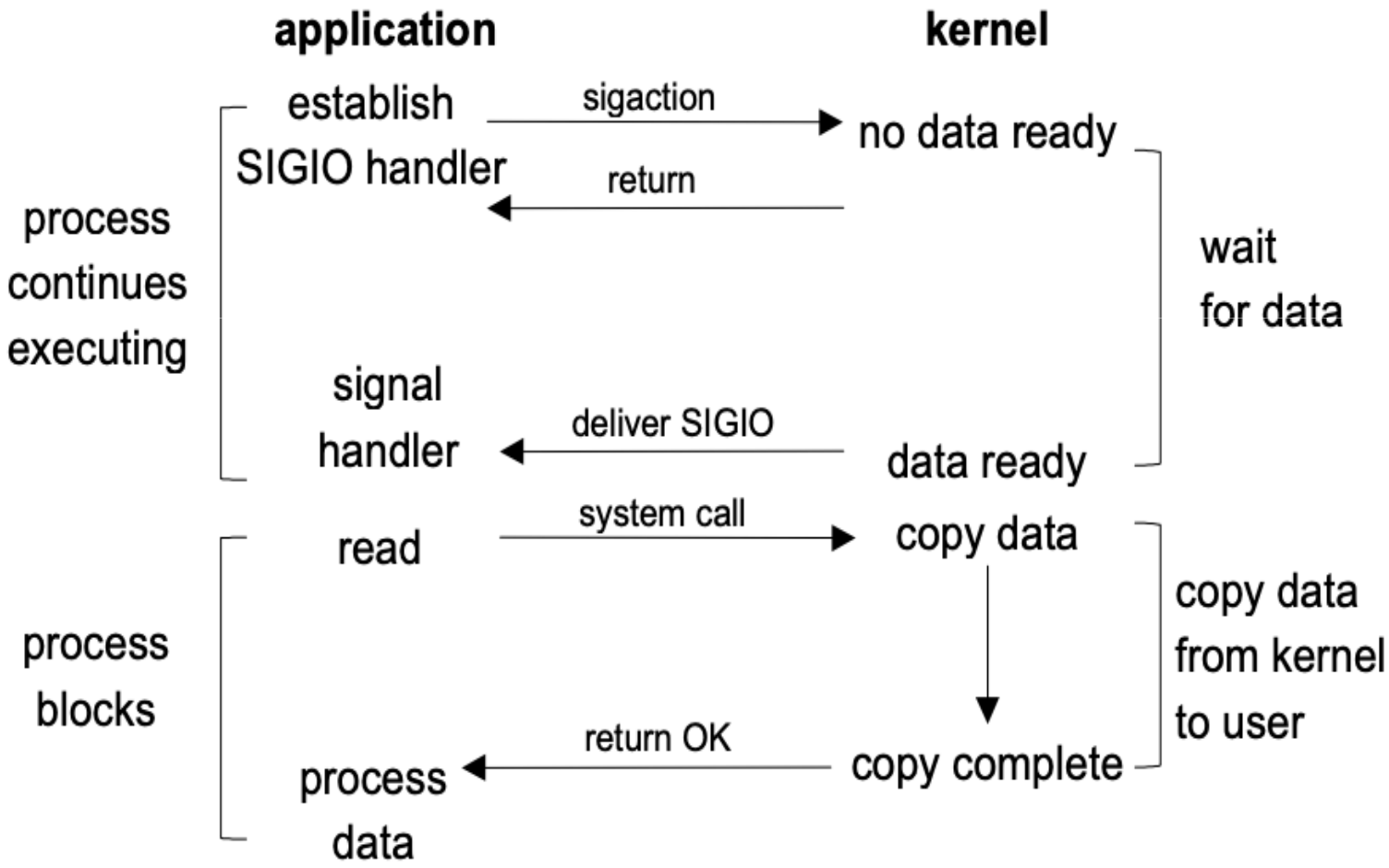


With **I/O multiplexing**, we call select or poll and block in one of these two system calls, instead of blocking in the actual I/O system call.

- We block in a call to select, waiting for the datagram socket to be readable.

- When select returns that the socket is readable, we then call recvfrom to copy the datagram into our application buffer.

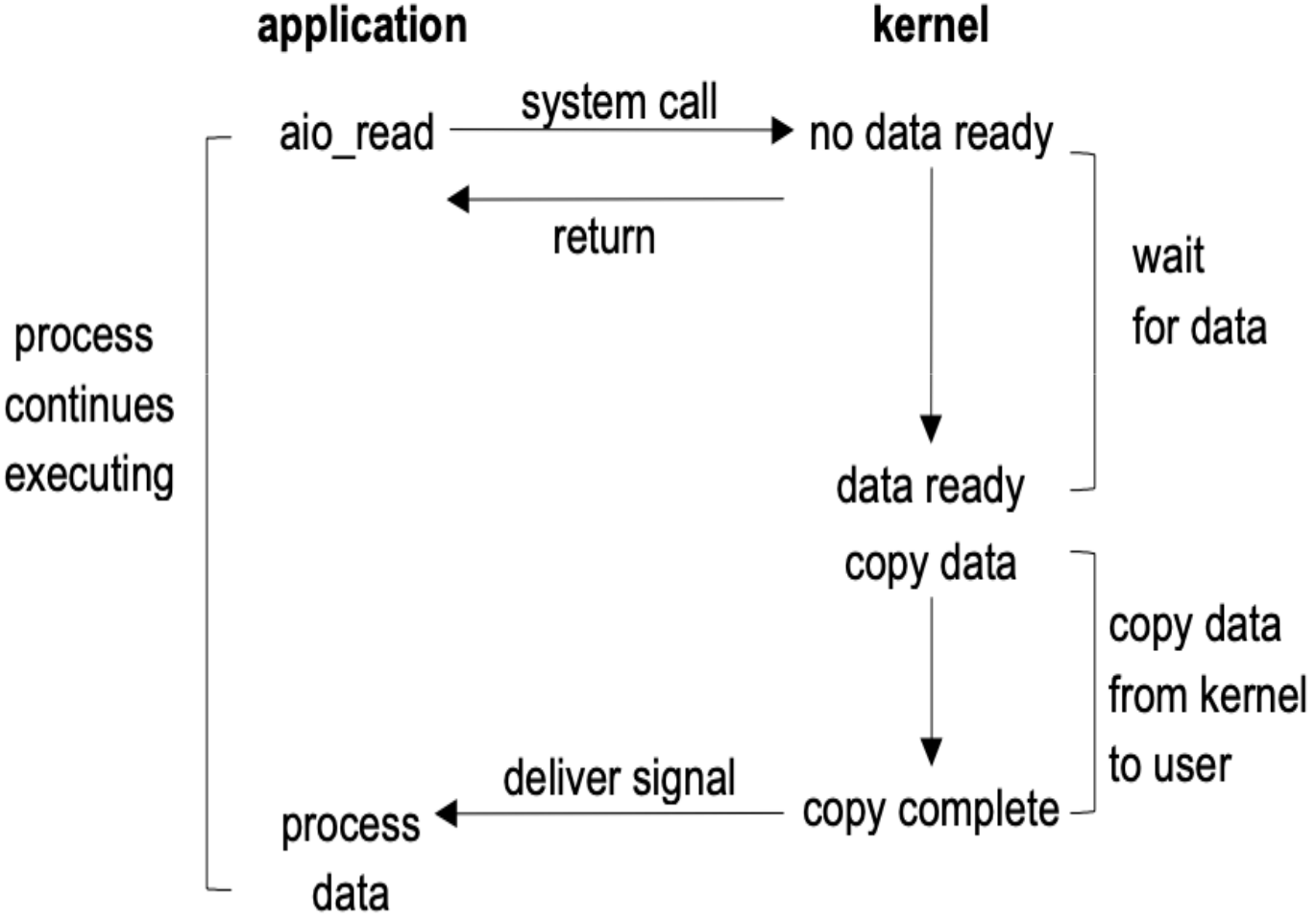
Signal Driven I/O Model



The **signal-driven I/O model** uses signals, telling the kernel to notify us with the SIGIO signal when the descriptor is ready.

- We first enable the socket for signal-driven I/O and install a signal handler using the **sigaction** system call.
- The return from this system call is immediate and our process continues; it is not blocked.
- When the datagram is ready to be read, the **SIGIO** signal is generated for our process.
- We can either:
  - read the datagram from the signal handler by calling recvfrom and then notify the main loop that the data is ready to be processed
  - notify the main loop and let it read the datagram.
- The advantage to this model is that we are not blocked while waiting for the datagram to arrive.
- The main loop can continue executing and just wait to be notified by the signal handler that either the data is ready to process or the datagram is ready to be read.

Asynchronous I/O Model

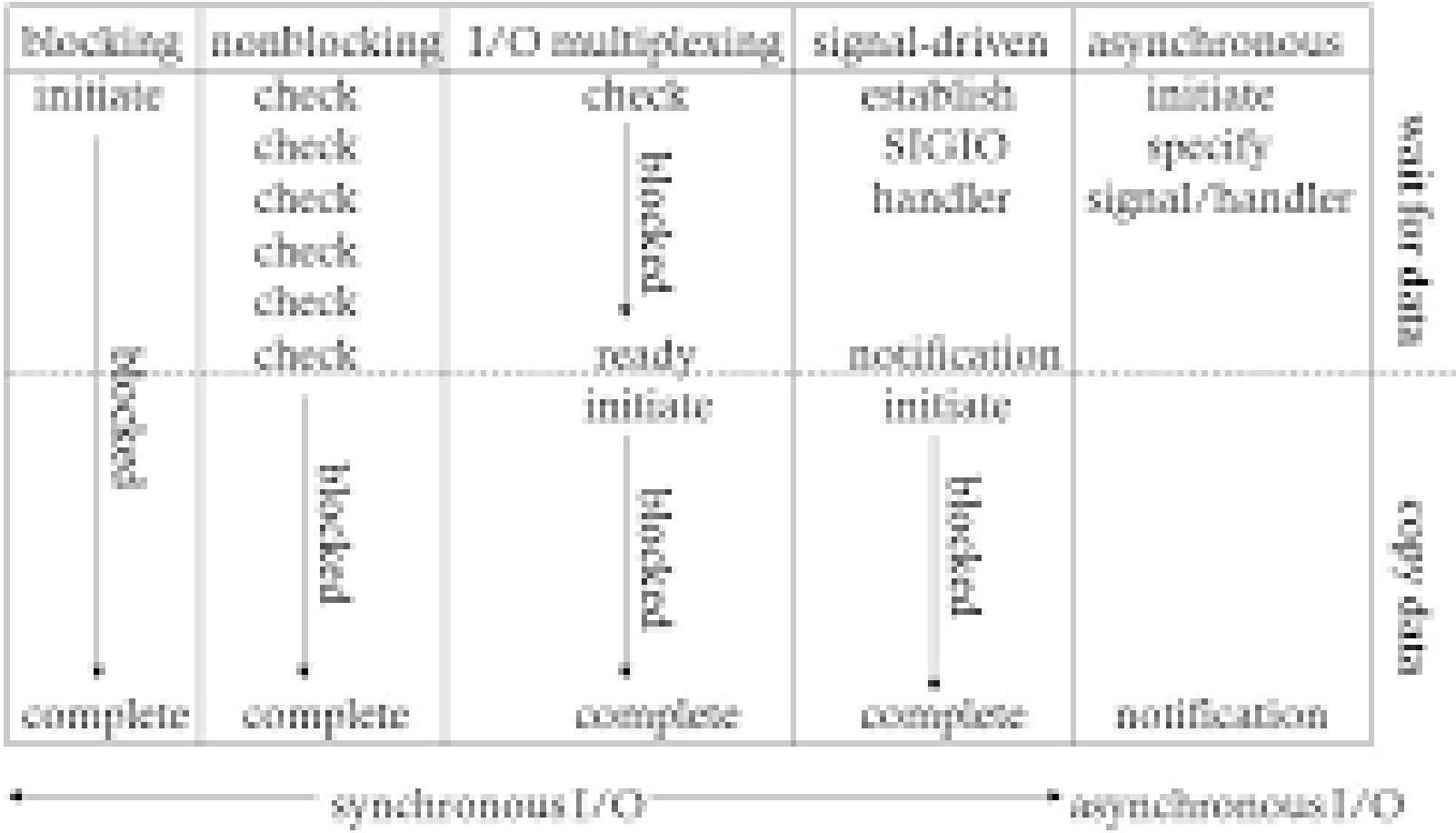


- These functions work by telling the kernel to start the operation and to notify us when the entire operation (including the copy of the data from the kernel to our buffer) is complete.
- The main difference between this model and the signal-driven I/O model is that with signal-driven I/O, the kernel tells us when an I/O operation can be initiated, but with asynchronous I/O, the kernel tells us when an I/O operation is complete.

- We call aio_read (the POSIX asynchronous I/O functions begin with aio_ or lio_) and pass the kernel the following:
  - descriptor, buffer pointer, buffer size (the same three arguments for read),
  - file offset (similar to lseek),
  - and how to notify us when the entire operation is complete.
- This system call returns immediately and our process is not blocked while waiting for the I/O to complete.
- We assume in this example that we ask the kernel to generate some signal when the operation is complete.
- This signal is not generated until the data has been copied into our application buffer, which is different from the signal-driven I/O model.

Comparison of the I/O Models

Comparison of Five I/O Models



- The main difference between the first four models is
- the first phase, as the second phase in the first four models is the same: the process is blocked in a call to recvfrom while the data is copied from the kernel to the caller's buffer.

- Asynchronous I/O, however, handles both phases and is different from the first four.

Synchronous I/O versus Asynchronous I/O

- POSIX defines these two terms as follows:
- A synchronous I/O operation causes the requesting process to be blocked until that I/O operation completes.
- An asynchronous I/O operation does not cause the requesting process to be blocked.
- Using these definitions, the first four I/O models (blocking, nonblocking, I/O multiplexing, and signal-driven I/O) are all synchronous because the actual I/O operation (recvfrom) blocks the process. Only the asynchronous I/O model matches the asynchronous I/O definition.

Select function

The select function allows the process to instruct the kernel to either:

- Wait for any one of multiple events to occur and to wake up the process only when one or more of these events occurs, or
- When a specified amount of time has passed.

This means that we tell the kernel what descriptors we are interested in (for reading, writing, or an exception condition) and how long to wait.

```
#include <sys/select.h>
#include <sys/time.h>
int select (int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset,
            const struct timeval *timeout);

   /* Returns: positive count of ready descriptors, 0 on timeout, –1 on error
*/
```

The ***timeout*** argument

The *timeout* argument tells the kernel how long to wait for one of the specified descriptors to become ready.

There are three possibilities for the *timeout*:

- **Wait forever** (*timeout* is specified as a null pointer). Return only when one of the specified descriptors is ready for I/O.
- **Wait up to a fixed amount of time** (*timeout* points to a timeval structure). Return when one of the specified descriptors is ready for I/O, but do not wait beyond the number of seconds and microseconds specified in the timeval structure.
- **Do not wait at all** (*timeout* points to a timeval structure and the timer value is 0, i.e. the number of seconds and microseconds specified by the structure are 0). Return immediately after checking the descriptors. This is called **polling**.

The descriptor sets arguments *

- *readset*
- *writeset*
- *exceptset*

- specify the descriptors that we want the kernel to test for reading, writing, and exception conditions.

- select uses **descriptor sets**, typically an array of integers, with each bit in each integer corresponding to a descriptor.
- datatype is **fd_set**

macros:

```
void FD_ZERO(fd_set *fdset);   /*clear all bits in   fdset */
void FD_SET(int fd, fd_set *fdset);  /* turn on the bit for fd
                                        in fdset */
void FD_CLR(int fd, fd_set *fdset);  /* turn off the bit for fd in
                                        fdset */
int  FD_ISSET(int fd, fd_set *fdset); /* is the bit for fd on in
                                        fdset ? */
```

Example

- fd_set rset;

- FD_ZERO(&rset); /* initialize the set: all bits off */
- FD_SET(1, &rset); /* turn on bit for fd 1 */
- FD_SET(4, &rset); /* turn on bit for fd 4 */
- FD_SET(5, &rset); /* turn on bit for fd 5 */

Any of the middle three arguments to select, *readset*, *writeset*, or *exceptset*, can be specified as a null pointer if we are not interested in that condition.

The ***maxfdp1*** argument

- It specifies the number of descriptors to be tested.
- Its value is the maximum descriptor to be tested plus one.
- The descriptors 0, 1, 2, up through and including *maxfdp1*–1 are tested.
- The constant **FD_SETSIZE**, defined by including <**sys/select.h**>, is the number of descriptors in the fd_set datatype.
- Its value is often 1024, but few programs use that many descriptors.

readset, *writeset*, and *exceptset* as value-result arguments *

- select modifies the descriptor sets pointed to by the *readset*, *writeset*, and *exceptset* pointers. These three arguments are value-result arguments.
- When we call the function, we specify the values of the descriptors that we are interested in, and on return, the result indicates which descriptors are ready.
- We use the FD_ISSET macro on return to test a specific descriptor in an fd_set structure.
- Any descriptor that is not ready on return will have its corresponding bit cleared in the descriptor set.
- To handle this, we turn on all the bits in which we are interested in all the descriptor sets each time we call select.

Return value of select

- The total number of bits that are ready across all the descriptor sets.

- If the timer value expires before any of the descriptors are ready, a value of 0 is returned.

- A return value of –1 indicates an error (which can happen, for example, if the function is interrupted by a caught signal).

str_cli Function with a select function

- The problem with earlier version of the str_cli () was that we could be blocked in the call to fgets when something happened on the socket.
- The client process is notified as soon as the server process terminates.
- The client process blocks in a call to select waiting for either standard input or the socket to be readable.

Three conditions are handled with the socket:

- If the peer TCP sends data, the socket becomes readable and read returns greater than 0 (the number of bytes of data).
- If the peer TCP sends a FIN (the peer process terminates), the socket becomes readable and read returns 0 (EOF).
- If the peer TCP sends an RST (the peer host has crashed and rebooted), the socket becomes readable, read returns –1, and errno contains the specific error code.

This code does the following:

- **Call** select.
  - We only need one descriptor set (rset) to check for readability. This set is initialized by FD_ZERO and then two bits are turned on using FD_SET:
  - ➢ the bit corresponding to the standard I/O file pointer, fp,
  - ➢ and the bit corresponding to the socket, sockfd.
  - The function fileno converts a standard I/O file pointer into its corresponding descriptor, since select (and poll) work only with descriptors.
  - select is called after calculating the maximum of the two descriptors.

- **Handle readable socket**. On return from select, if the socket is readable, the echoed line is read with readline and output by fputs.
- **Handle readable input**. If the standard input is readable, a line is read by fgets and written to the socket using writen.

- Instead of the function flow being driven by the call to fgets, it is now driven by the call to select

shutdown Function

```
#include <sys/socket.h>
int shutdown(int sockfd, int howto);

/* Returns: 0 if OK, –1 on error */
```

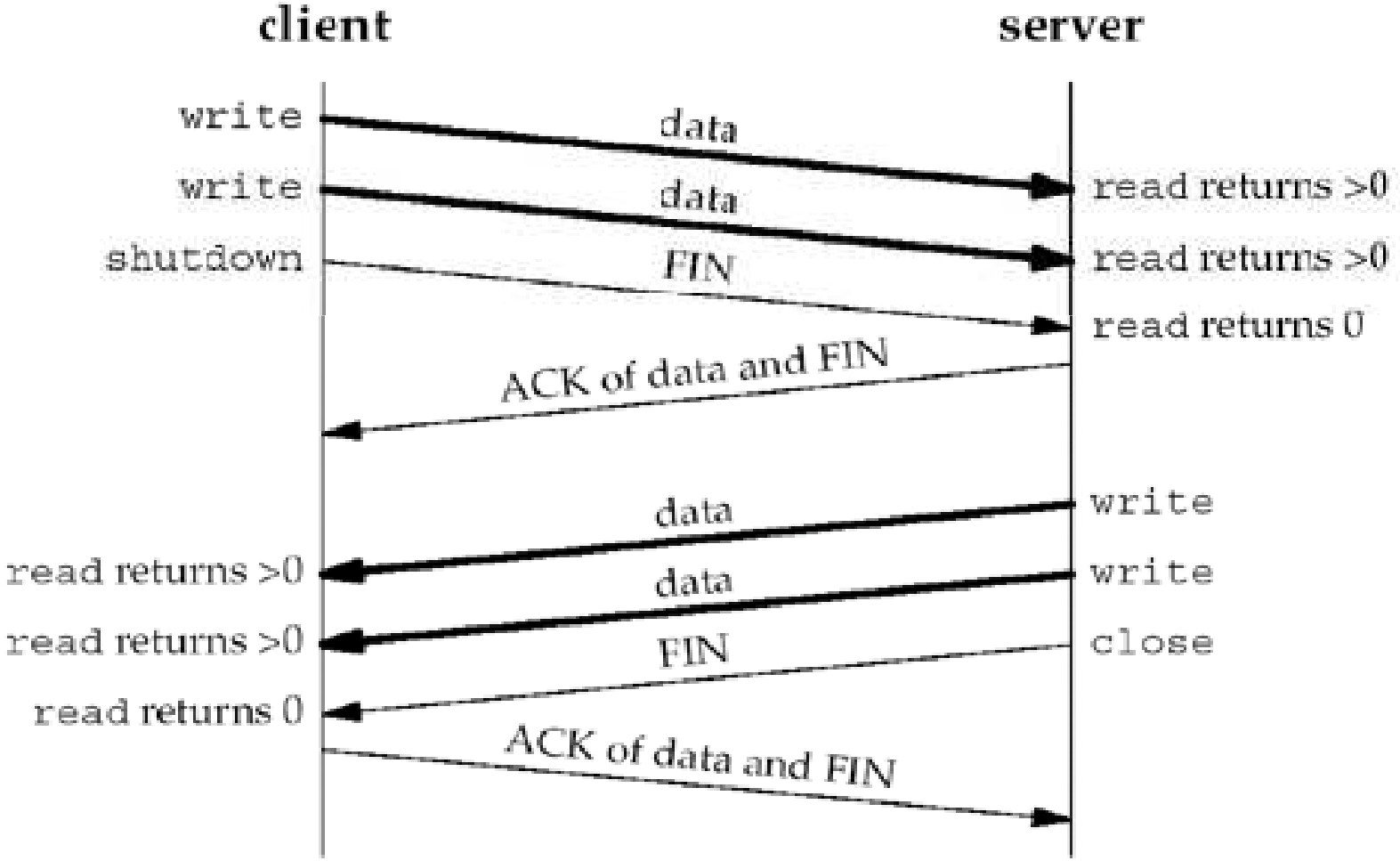
shutdown Function

- Normal termination of a function uses **close ()**.
- There are two limitations with close that can be avoided with shutdown:

➢ close decrements the descriptor's reference count and closes the socket only if the count reaches 0. With shutdown, we can initiate TCP's normal connection termination sequence (the four segments beginning with a FIN in, regardless of the reference count).

➢ close terminates both directions of data transfer, reading and writing. Since a TCP connection is full-duplex, there are times when we want to tell the other end that we have finished sending, even though that end might have more data to send us (batch input).

Calling shutdown to close half of a TCP connection



The action of the function depends on the value of the ***howto*** argument:

- SHUT_RD: The read half of the connection is closed.

- SHUT_WR: The write half of the connection is closed**.** In the case of TCP, this is called a **half-close**. Any data currently in the socket send buffer will be sent, followed by TCP's normal connection termination sequence. The process can no longer issue any of the write functions on the socket.

- SHUT_RDWR: The read half and the write half of the connection are both closed. This is equivalent to calling shutdown twice: first with SHUT_RD and then with SHUT_WR.

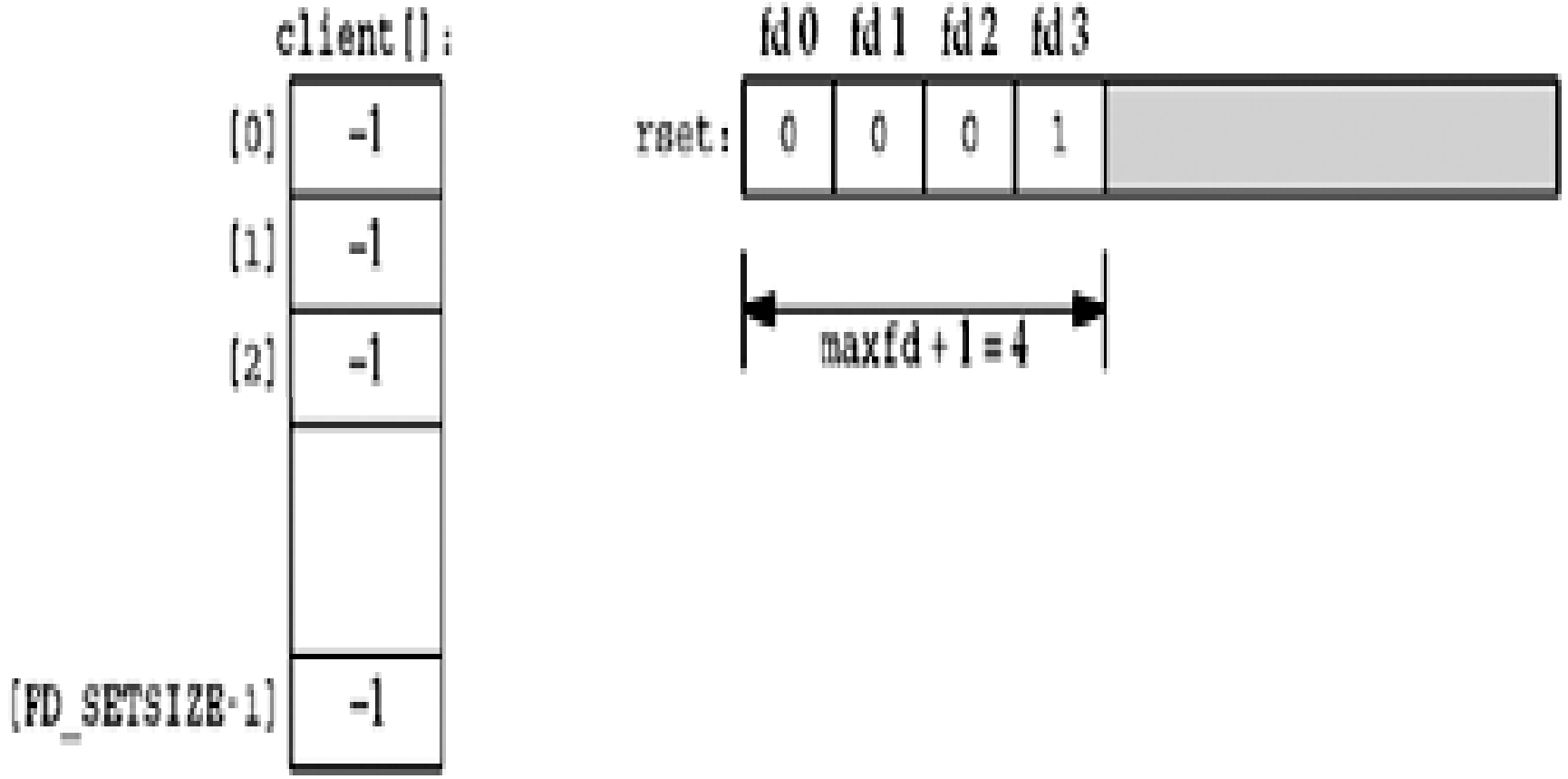
The three SHUT_*xxx* names are defined by the POSIX specification. Typical values for the howto argument that you will encounter will be 0 (close the read half), 1 (close the write half), and 2 (close the read half and the write half).

TCP Echo Server (Revisited)

TCP echo server as a single process that uses select to handle any number of clients, instead of forking one child per client.

Before first client has established a connection

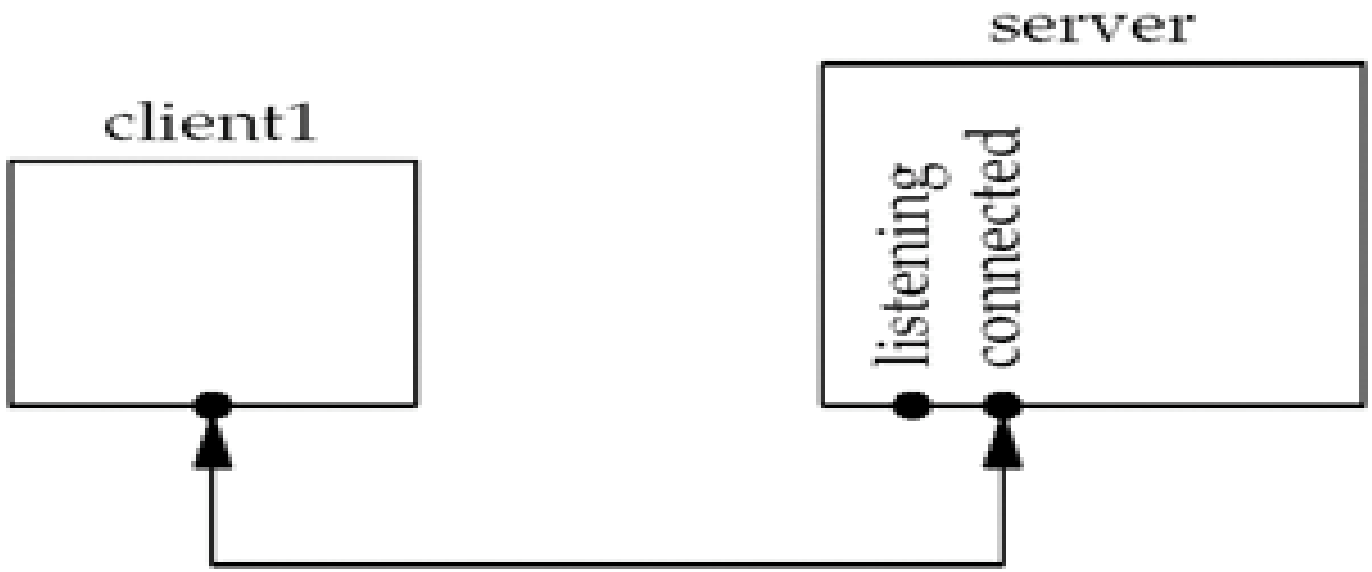
- the server has a single listening descriptor.
- The server maintains only a read descriptor set (*rset*).
- Descriptors 0, 1, and 2 are set to standard input, output, and error, so the first available descriptor for the listening socket is 3.
- An array of integers named client that contains the connected socket descriptor for each client. All elements in this array are initialized to –1.



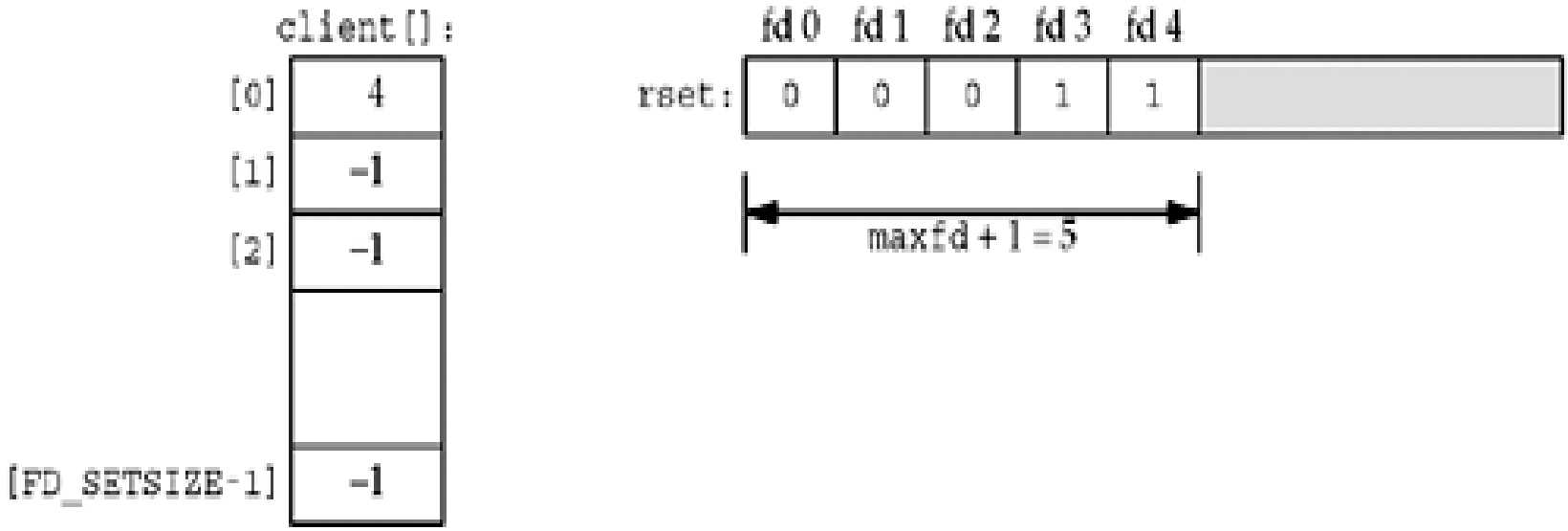
The only nonzero entry in the descriptor set is the entry for the listening sockets and the first argument to `select` will be 4.

After first client establishes connection

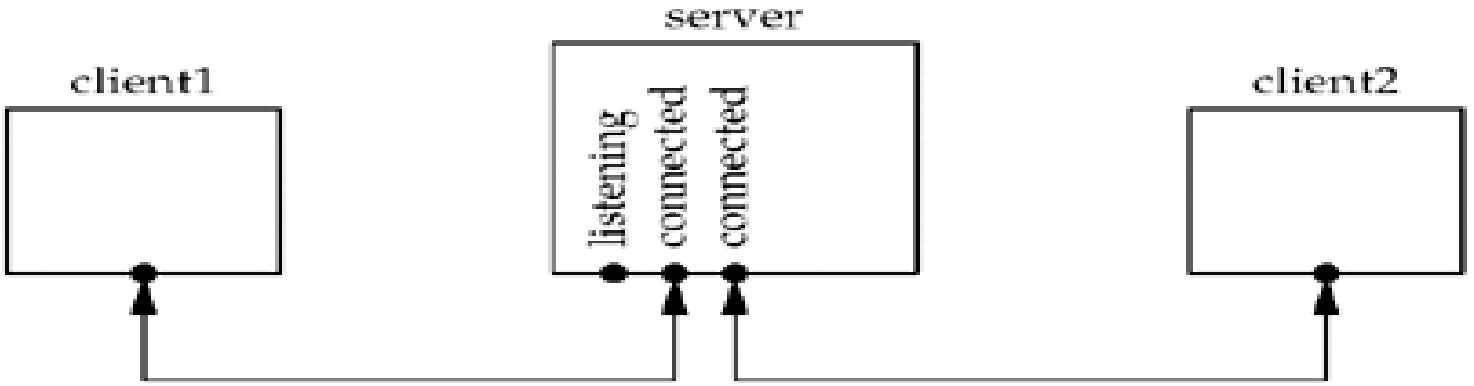
- the listening descriptor becomes readable
- our server calls accept.
- The new connected descriptor returned by accept will be 4.



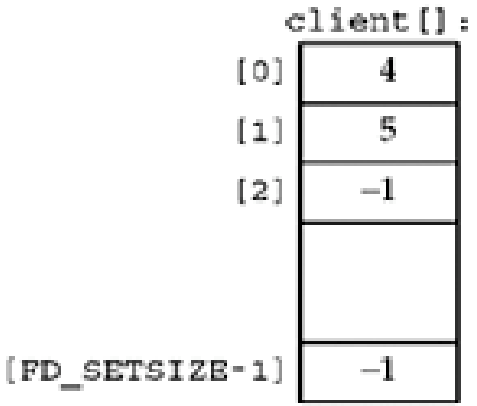
- The server must remember the new connected socket in its client array
- the connected socket must be added to the descriptor set.
- The updated data structures is:

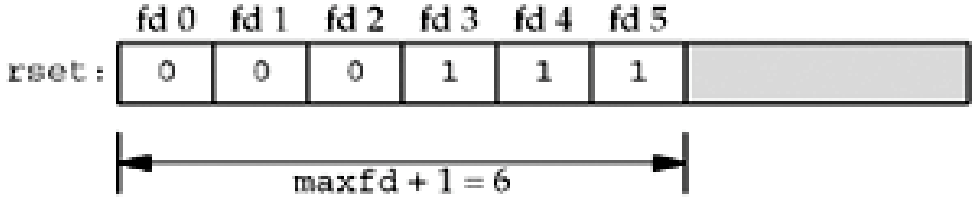


After second client connection is established *



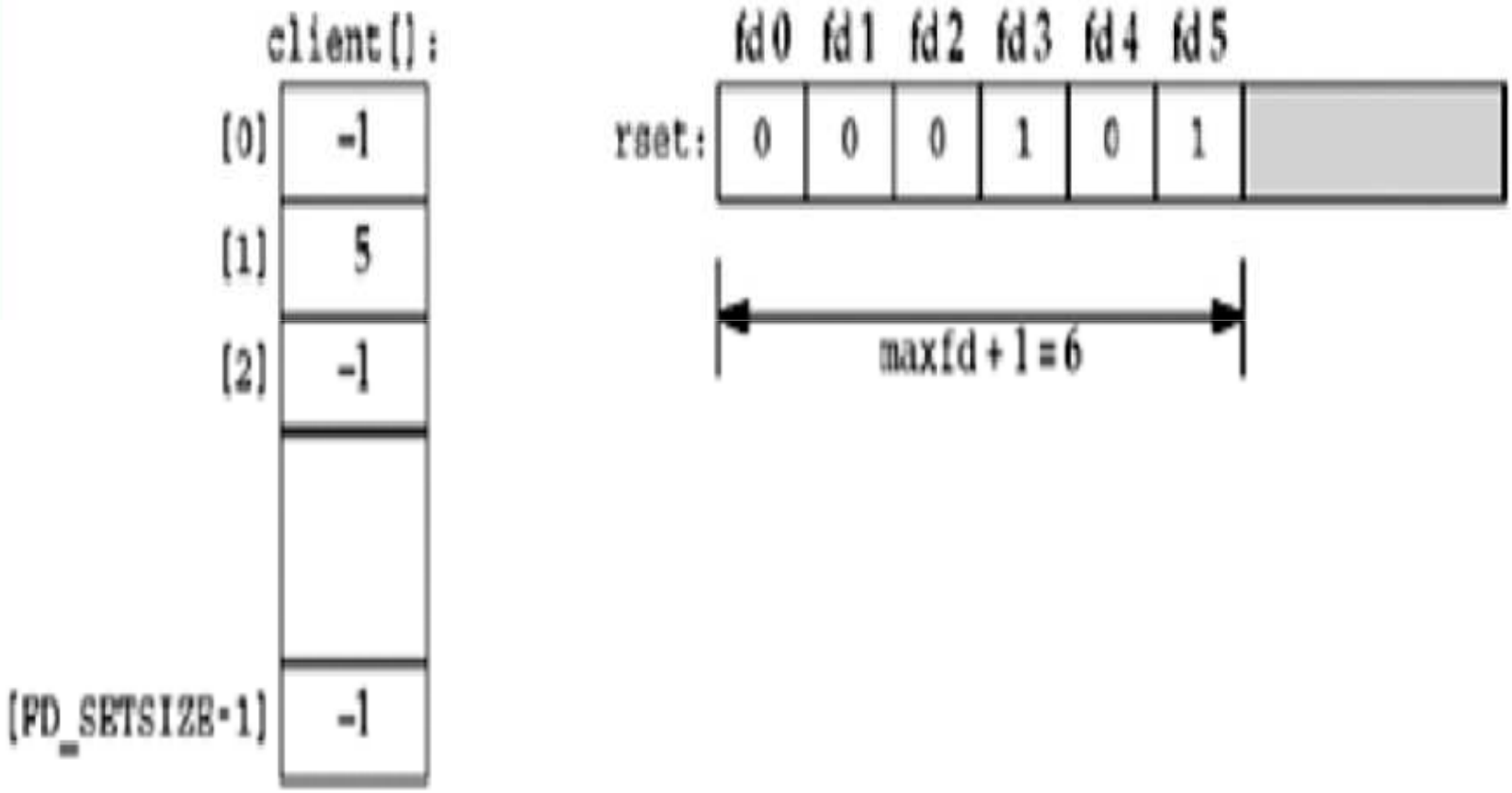
The new connected socket (which we assume is 5) must be remembered, giving the data structures





After first client terminates its connection *

- The client TCP sends a FIN, which makes descriptor 4 in the server readable.
- When the server reads this connected socket read returns 0.
- We this socket is then closed and the data structures are update accordingly.
- The value of client[0] is set to –1 and descriptor 4 in the descriptor set is set to 0.
- The value of maxfd does not change.


client[]:
[0]
-1
[1]
5
[2]
-1
[FD_SETSIZE-1]
-1
fd0 fd1 fd2 fd3 fd4 fd5
rset:
0 0 0 1 0 1
maxfd+1=6

- The code does the following:
- **Create listening socket and initialize for select.**
- We create the listening socket using **socket, bind,** and **listen** and initialize our data structures assuming that the only descriptor that we will **select** on initially is the listening socket.
- **Block in select.**
- **select** waits for something to happen, which is one of the following:
  - The establishment of a new client connection.
  - The arrival of data on the existing connection.
  - A FIN on the existing connection.
  - A RST on the existing connection.
-

- **accept new connections**.
  - If the listening socket is readable, a new connection has been established.
  - We call accept and update our data structures accordingly. We use the first unused entry in the client array to record the connected socket.
  - The number of ready descriptors is decremented, and if it is 0 (tcpcliserv/tcpservselect01.c#L62), we can avoid the next for loop. This lets us use the return value from select to avoid checking descriptors that are not ready.
- **Check existing connections**.
  - In the second nested for loop, a test is made for each existing client connection as to whether or not its descriptor is in the descriptor set returned by select, and a line is read from the client and echoed back to the client. Otherwsie, if the client closes the connection, read returns 0 and we update our data structures accordingly.
  - We never decrement the value of maxi, but we could check for this possibility each time a client closes its connection.